Question 049 | Question bank page 5 | Q.1

Develop a College Information Portal website and convert it into a Progressive Web Application (PWA) by implementing a Service Worker and Web App Manifest.

Project type: Progressive Web Application

Steps to perform

1. Create an ASP.NET Core MVC project or a suitable static web project in Visual Studio.

2. Add HTML or Razor pages, manifest.json, service-worker.js, and any referenced icons under wwwroot.

3. Register the service worker and link the manifest in the HTML layout.

4. Run the site on localhost or HTTPS and inspect it in a modern browser.

5. Use browser Developer Tools to check manifest, service worker, cache and offline operation. Push notifications require additional permission and subscription/server setup.

Closest supplied practical: Create a simple website for a College Information Portal and convert it into a Progressive Application.

Source: Dot_Net_Practicals_6_to_10_Organized_v2.pdf

Provided creation instructions and source files

```
Creation steps: Create an MVC project named CollegePortalPWA. Add the page, manifest and service worker
files; link the manifest and register the service worker in the layout; enable static files in
Program.cs.
Filename: Index.cshtml    Location: Views/Home/Index.cshtml
<div class="text-center">
    <h1>College Information Portal</h1>
    <h3>Welcome to xyz College</h3>
    <p>Courses Offered:</p>
    <ul>
        <li>BSC IT</li>
        <li>BSC CS</li>
        <li>BCA</li>
    </ul>
    <p>Contact: 9818763456</p>
</div>
Filename: manifest.json    Location: wwwroot/manifest.json
{
  "name": "College Information Portal",
  "short_name": "CollegePortal",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0d6efd",
  "icons": [
    {
      "src": "icons/icon1.png",
      "sizes": "192x192",
      "type": "image/png"
    },
    {
      "src": "icons/icon2.png",
      "sizes": "512x512",
      "type": "image/png"
    }
  ]
}
Filename: service-worker.js    Location: wwwroot/service-worker.js
self.addEventListener('install', function (event) {
    console.log('Service Worker Installed');
});
self.addEventListener('fetch', function (event) {
    console.log('Fetching:', event.request.url);
});
Filename: Layout    Location: Views/Shared/_Layout.cshtml
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - CollegePortalPWA3</title>
    <script type="importmap"></script>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
```

```
    <link rel="stylesheet" href="~/CollegePortalPWA3.styles.css" asp-append-version="true" />
    <link rel="manifest" href="~/manifest.json" />
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom
box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home"
asp-action="Index">CollegePortalPWA3</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target=".navbar-collapse" aria-
controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">
                    <span
 class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Privacy">Privacy</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>

    </header>
    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>
    <footer class="border-top footer text-muted">
        <div class="container">
            &copy; 2026 - CollegePortalPWA3 - <a asp-area="" asp-controller="Home"
asp-action="Privacy">Privacy</a>
        </div>
    </footer>
    <script src="~/lib/jquery/dist/jquery.min.js"></script>
    <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)
    <script>
        if ('serviceWorker' in navigator) {
            navigator.serviceWorker.register('/service-worker.js')
                .then((registration) => {
                    console.log('Service Worker Registered!', registration);
                })
                .catch((error) => {
                    console.log('Service Worker Registration Failed:', error);
                });
        }
    </script>
</body>
</html>
Filename: Program.cs    Location: Program.cs
Adding a line – app.UseStaticFiles();

Create a PWA that displays a custom offline page when the user is not connected to
```

Expected result / verification

The application should meet the exact question requirements. Record actual output or screenshots after running; no execution has been claimed in this manual.